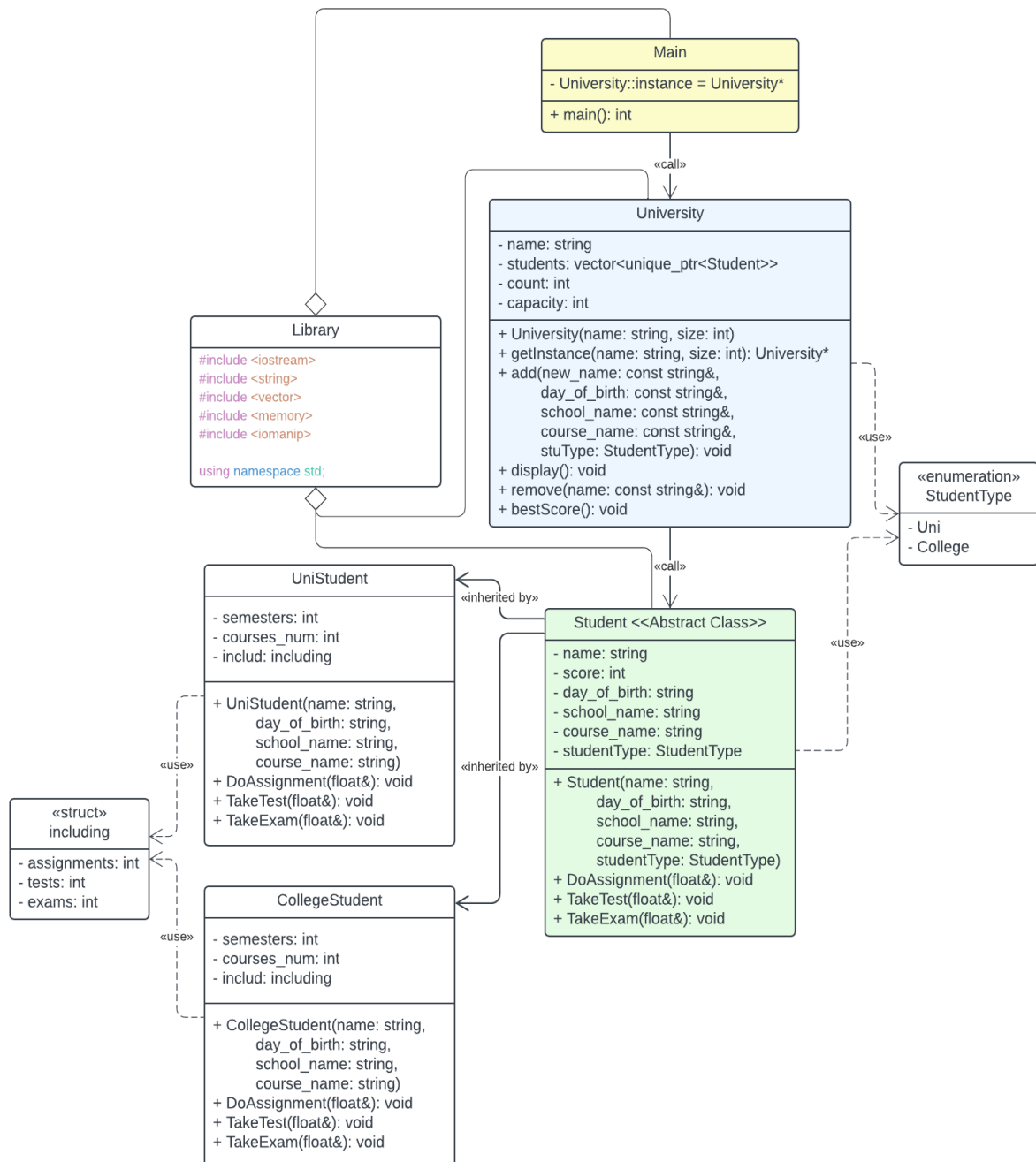


The UML diagram below illustrates the student management program written in C++.

Overall, the program consists of Class University designed as Pattern Factory; Abstract Class Student; two Inherited Class from Student which are UniStudent and CollegeStudent; and the main function. Enum class StudentType and Struct including are also implemented into the source code; along with including libraries, which are <iostream>, <string>, <vector>, <memory>, and <iomanip>.



This report will not break down the whole source code; instead, it will point out some of the adjustments and how would they improve the overall performance of the program.

1st Changes: Utilizing Struct and Vector data structures

As we know, Struct groups related variables together for making the code more organized and readable. In this case, there are three related variables in Student Class which determine the overall score of a student; they are number of assignments, tests, and exams. Struct including { int assignments; int tests; int exams; }; stores them into one data structure; instead of declaring multiple individual variables. This can lead to better memory saving, as the C++ compiler may optimize memory layout and alignment for the Struct.

```
// Struct used to store numbers of each part of a course that students must take
struct including {
    int assignments;
    int tests;
    int exams;
};
```

By encapsulating related data into a single unit, it provides a level of abstraction; which helps in managing complexity by hiding implementation details, such as number of assignments or tests that a student needs to accomplish, and exposing only the necessary information. Especially when it is included in two inherited Classes UniStudent and CollegeStudent, which have different number of tasks that need to be done.

<pre>class UniStudent : public Student { private: int semesters; int courses_num; struct including includ { 3, 2, 1 }; };</pre>	<pre>class CollegeStudent : public Student { private: int semesters; int courses_num; struct including includ { 1, 1, 1 }; };</pre>
---	---

Next, let go with the list of students inside of Class University. On the left-hand side below, Vector of Unique Pointers is utilized for managing students in a university, instead of 2D dynamical array on the right-hand side. This is because Vector will automatically manage memory allocation and resizing; allowing to add or remove elements dynamically without worrying about memory management, and the chances of memory leaks and buffer overflows. It also offers functions like push_back(), pop_back(), resize(), insert(), and erase() for easier dynamic manipulation of elements.

```
class University {
private:
    string name;
    vector<unique_ptr<Student>> students;
```

```
class University {
private:
    string name;
    Student** students;
```

The benefits of using Unique Pointers in Vector will be discussed later in “3rd Changes”.

2nd Changes: Utilizing Design Pattern Singleton and Factory data structures

Class University in this case acts as tool which is expected to effectively manage all students. Therefore, it cannot be duplicated or wasted; and all of its modifications should be consistently reflected everywhere in the whole program. In order to archive this, it is needed to be a single instance which can be accessed

by any components within the codebase. The Singleton Design Pattern is a perfect solution, as is used to ensure a class only has one instance and provides a global point of access to that instance.

The following source code below show how Class University is declared as a Singleton instance.

```
class University {
private:
    string name;
    vector<unique_ptr<Student>> students; // Vector of unique pointers for managing students in a university
    int capacity;
    int count = 0;

    // Private constructor to prevent direct instantiation
    University(string name, int size) : name(name), capacity(size) {
        students.reserve(size);
    }

    // Static instance of the University class
    static University* instance;
public:
    // Static method to get the singleton instance
    static University* getInstance(string name, int size) {
        if (!instance) {
            instance = new University(name, size);
        }
        return instance;
    }

    // Disable copy constructor and assignment operator
    University(const University&) = delete;
    University& operator=(const University&) = delete;

    // Destructor to clean up the instance
    ~University() {
        delete instance;
    }
};

// Initialize static instance to nullptr
University* University::instance = nullptr;
```

First, its Constructor is defined as Private to prevent direct instantiation.

```
// Private constructor to prevent direct instantiation
University(string name, int size) : name(name), capacity(size) {
    students.reserve(size);
}
```

A Static variable instance of the Class is also defined.

```
// Static instance of the University class
static University* instance;
```

Then, the program declares getInstance() function to get the instance of University Class when called. It returns a pointer to University object. The if statement checks if the instance has been initialized or not.

The code block inside of the if statement can only be executed if the instance has not been initialized yet (instance == nullptr). This make sure that only one instance of Class University can be created.

```
public:
    // Static method to get the singleton instance
    static University* getInstance(string name, int size) {
        if (!instance) {
            instance = new University(name, size);
        }
        return instance;
    }
}
```

The lines below are used to prevent copying and assigning an initialized instance to another instance of the same University class. By using delete, the Class cannot be copied or its object cannot be assigned value of another object of the same type by using standard syntax.

```
// Disable copy constructor and assignment operator
University(const University&) = delete;
University& operator=(const University&) = delete;
```

A Destructor is required to delete the instance; since it is the pointer which points to the University object.

```
// Destructor to clean up the instance
~University() {
    delete instance;
}
```

Finally, the instance is declared outside of the Class as nullptr.

```
// Initialize static instance to nullptr
University* University::instance = nullptr;
```

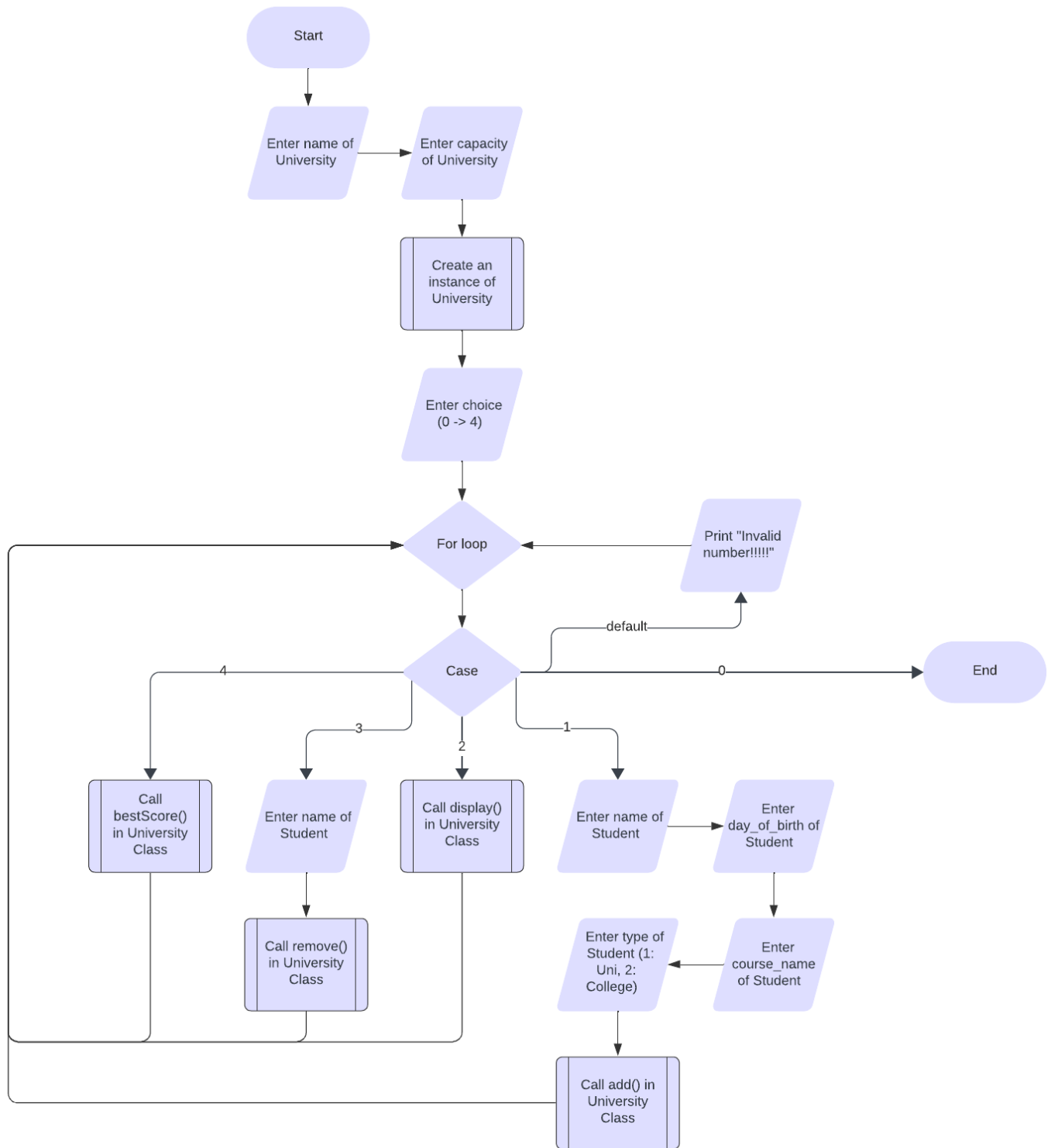
Class University is also designed as a Student Factory which stores the information of two distinctive types of Student (UniStudent and CollegeStudent). Such characteristic is mostly illustrated in its add() function.

```
void add(const string& new_name, const string& day_of_birth, const string& school_name,
        const string& course_name, StudentType stuType) {
    if (count >= capacity) {
        cout << "The list is full!!\n";
        return;
    }

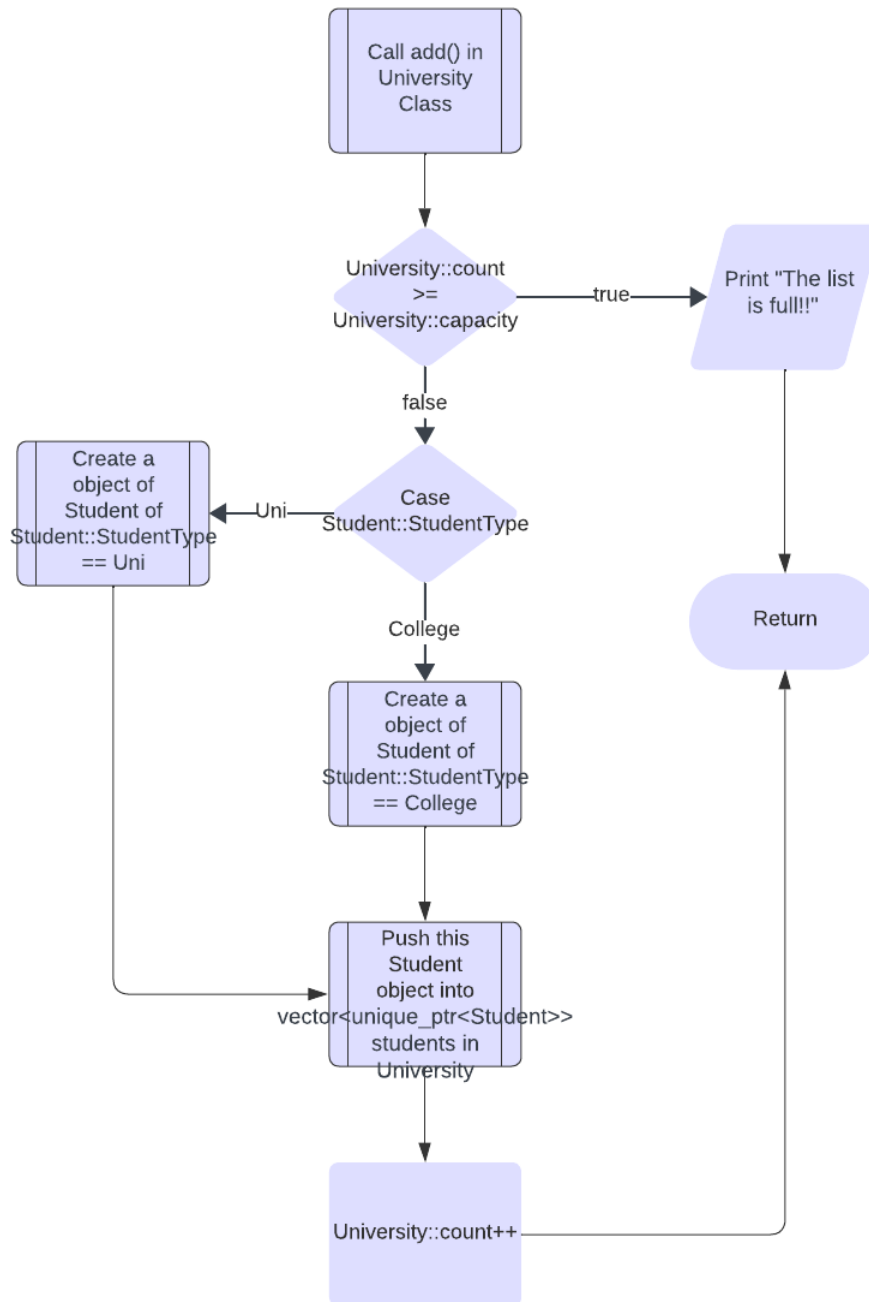
    if (stuType == StudentType::Uni)
        students.push_back(make_unique<UniStudent>(new_name, day_of_birth, school_name, course_name));
    else if (stuType == StudentType::College)
        students.push_back(make_unique<CollegeStudent>(new_name, day_of_birth, school_name, course_name));

    ++count;
}
```

The flowchart below shows that the user will have to enter types of Student first, calling add() function.



In the add() function, instances of different Student can be directly created; thus, tightly links Class University with specific Student types. This ensures better encapsulation and provides simplification in the client code (Class University), by making it responsible for managing students, instead of detailed concerning of creating them.



Introducing Factory helps utilizing better abstraction of the process of creating Student object; as University class interacts with the factory interface rather than directly with concrete student classes (UniStudent and CollegeStudent). As a result, when removing a student from the list, display the whole list or list of students with best scores, the program only has to traverse through each student by iterator representing Class Student's object without recognizing specific implementation details of each student type as an object of UniStudent or CollegeStudent.

Here is the example of how students of type Student in the list are displaying.

```
// Range for loop is implemented to print all student's details in students vector
for (const auto& student : students) {
    cout << student->name << " - " << student->day_of_birth <<
        " - " << student->course_name << " - " <<
        ((student->studentType == StudentType::Uni) ? "Uni" : "College") <<
        " - " << student->score << endl;
}
cout << endl;
```

And here is how Student in the list is searched and deleted if matched.

```
// The use of auto helps to probably initiate suitable data type for variable it
// Lamda function to check for each student's name if it is matched or not
auto it = find_if(students.begin(), students.end(),
    [&](const unique_ptr<Student>& student) {return (student->name == name);});

if (it != students.end()) {
    students.erase(it);
    --count;
}
else {
    cout << "Cannot find a student with that name!!\n";
}
```

Lastly, the for loop below finds the score gained by one or more students in the list.

```
float max = 0;
for (const auto& student : students) {
    if (student->score > max) {
        max = student->score;
    }
}
```

From the following examples, we can see that the University class can modify and track of all its students in the most flexible and convenient way,, which provides easier later testing. The following source codes are also utilizing Modern Auto, Range for Loop, and Lamda function; which will be discussed in “3rd Changes”.

3rd Changes: Utilizing Modern Smart Pointer, Range for Loop, Auto, and Lamda expression

Along with the use of Vector, Smart Pointer automatically handle memory management, including memory deallocation when objects are no longer needed; which helps prevent memory leaks if memory is not properly deallocated. Its only role is to manage the ownership of Student objects stored in the students Vector of University class, ensuring that memory is properly managed throughout the program's lifecycle.

The use of Unique Pointer ensures that each object is owned by exactly one smart pointer. This ownership model prevents the creation of dangling pointers, which can occur when raw pointers are used and an object is deleted while other pointers still reference it. The example below shows how each Student object is created and pushed back into the Student list.

```
if (stuType == StudentType::Uni)
    students.push_back(make_unique<UniStudent>(new_name,
        day_of_birth, school_name, course_name));
else if (stuType == StudentType::College)
    students.push_back(make_unique<CollegeStudent>(new_name,
        day_of_birth, school_name, course_name));
```

When retrieving a student information or removing a student in the list, the use of Modern Range for Loop automatically handles the iteration over the entire range of students in a container, eliminating the risk of off-by-one errors (when violating boundary of loops) that are common with traditional loop (as shown below).

```
for (int i = 0; i < count; ++i) {
    cout << getList()[i]->name << " - " << getList()[i]->day_of_birth <<
        " - " << getList()[i]->course_name << " - " <<
        ((getList()[i]->studentType == 0) ? "Uni" : "College") <<
        " - " << fixed << setprecision(2) << getList()[i]->score << endl;
}
```

As we can see below, with Modern Range for Loop, we do not need to set up a limitation or break statement in a loop. Auto type is also used for automatic type inference, making the code more concise and less error-prone, especially when dealing with assigning complex or nested container types. In this case, student will be declared as a Pointer of <unique_ptr<Student>, without writing out a suitable data type.

```
// Range for loop is implemented to print all student's details in students vector
for (const auto& student : students) {
    cout << student->name << " - " << student->day_of_birth <<
        " - " << student->course_name << " - " <<
        ((student->studentType == StudentType::Uni) ? "Uni" : "College") <<
        " - " << student->score << endl;
}
cout << endl;
```


Here is another example of utilizing Auto to probably initiate iterator it. Along with the use of Auto, the implementation of Lamda expression [&](const unique_ptr<Student>& student) {return (student->name == name);} ensures more compact and expressive way to define a function inline within the code; as it is unnecessary to design the whole traditional function to check for a student with the matching name, and place its return value as a predicate for find_if() function. The [&] inside of Lamda structure captures all variables by reference from the enclosing scope, so the function can capture the name variable from the surrounding scope.

```
// The use of auto helps to probably initiate suitable data type for variable it
// Lamda function to check for each student's name if it is matched or not
auto it = find_if(students.begin(), students.end(),
    [&](const unique_ptr<Student>& student) {return (student->name == name);});
```

In summary, the 3rd adjustments are the most crucial in all three modification stages; due to its direct effectiveness to the operations of displaying students' data, delete student, or searching for best students inside of University. Those are implemented to satisfy the approaches as shown as flowcharts below.

